

오늘의 작업기록 — 2026년 7월 17일

m106 착수 — CodeCrafters 계정 연결 + clone 명령 확보

(수련회 첫날, 둘째 이월 대신 맨 앞 조각만 떼서 완결한 날)

한눈에

항목	내용
상황	수련회(7/17~19) 첫날 — 시간 거의 없음, m106은 이미 두 번 이월된 상태
판단	세 번째 둘째 이월 대신 착수 조각으로 쪼갬 — 완결점: clone 명령 확보
한 일	codecrafters.io → GitHub 로그인 → Build Your Own Redis → 언어 Java → clone 명령 확보 ✓
실질 근거	내일의 최대 미지수(모바일 UI 탐색, 8~12분 구간)가 오늘 제거됨 — 내일은 터미널에서 시작
내일용 주의 2	① 기본 브랜치 master(main 아님) ② CodeCrafters push는 GitHub 잔디에 안 찍힘(그들 git 서버)
오늘 잔디	이 작업기록 파일의 logs 커밋 (확립 경로 — m106 기간 내내 이 경로가 잔디다)
내일 플랜	20분: clone+정찰+before / 40분+: Stage 1 초록 + m106 done 완주

1. 오늘의 판단 — 쪼개는 기준은 "내일이 어디서 시작하나"

m106은 7/15, 7/16 두 번 이월됐다. 오늘도 시간이 없으니 세 번째 이월이 자연스러워 보이지만 — 둘째 이월과 착수 조각은 다르다. 어제 세운 예상 시간표를 다시 보면:

구간	예상	미지수
1. 계정+시작 (폰 브라우저)	8~12분	최대 — 모바일 최적화 안 된 사이트에서 버튼 찾기
2~6. 클론~완주 (터미널)	25~35분	작음 — 전부 익숙한 도구(git, nano, push)

미지수가 맨 앞 구간에 몰려 있다. 그럼 그 구간만 오늘 떼면, 내일 세션은 미지수 없이 익숙한 터미널 작업으로만 구성된다. 쪼개는 기준이 "오늘 몇 분 확보하나"가 아니라 "내일 세션이 어디서 시작하게 되나"였던 것 — 오늘의 5~10분이 내일의 첫 10분(가장 느린 구간)을 지웠다.

완결점을 "계정"이 아니라 "clone 명령 확보"로 잡은 이유

clone 명령은 계정만 만들면 나오는 게 아니다 — 코스(Redis)와 언어(Java)를 선택해야 발급된다. 계정만 만들고 멈추면 내일 다시 브라우저 UI를 헤매야 하니 반쪽이다. "여기서 멈추면 내일 뭐부터 하지?"에 "clone 한 줄 실행"이라고 답할 수 있는 지점 — 거기가 완결점이다.

2. 한 일 — 절차 기록

- 폰 브라우저에서 codecrafters.io → GitHub으로 로그인 (Taewoo-Won 계정, Authorize)
- 코스 목록에서 Build Your Own Redis 선택
- 언어 Java 선택 (무료 트랙, 카드 불필요)
- 화면에 발급된 git clone https://git.codecrafters.io/... 명령 확보 ← 오늘의 완결점

새 개념은 없다 — 의도된 것이다. ServerSocket, accept() 같은 오늘의 몹이었던 개념들은 내일 코드에서 처음 만나는 게 맞다(정찰 단계에서 읽으며 해제하는 설계라서, 개념과 코드가 같은 순간에 오는 게 이 코스의 방식이다).

내일용 주의사항 둘 (어제 플랜에서 옮겨 박음)

- 기본 브랜치가 master다 — push는 git push origin master . main으로 치면 거부된다.
- CodeCrafters push는 GitHub 잔디에 안 찍힌다 — 그들의 git 서버(git.codecrafters.io)로 가기 때문. m106 기간의 잔디는 작업기록 파일이 backend-journey/logs 에 커밋되는 확립 경로다. 오늘 잔디도 이 파일이다.

3. 내일(7/18) 플랜 — 시간 규모별 끊는 지점

20분이면 (여기까지도 완결 조각):

```
cd ~ && git clone <확보한 주소>
cd <생긴 폴더> && cat README.md && cat src/main/java/Main.java # 정찰
git commit --allow-empty -m "run tests" && git push origin master # before — 실패 먼저
```

→ 터미널에 스트림되는 테스트 결과에서 "port 6379 연결 불가" 실패를 눈으로 확인하는 것까지. (before 증거 — 확립된 스타일 그대로.)

40분+면 완주: - Main.java 주석 해제 — 각 줄 읽으면서. 특히 accept() 한 줄: 7/6 Q1의 "서버의 존재 이유는 기다림"에서 그 '기다림'의 정체가 코드로 처음 보이는 순간이다. - git commit -am "pass stage 1" && git push origin master → Stage 1 초록 - (선택 5분) ./your_program.sh 띄우고 다른 pane에서 ss -tlnp | grep

6379 — 내가 연 포트를 내 판정 도구로 본다. 끝나면 내리고 ss 빈 출력(공개 IP 규율, 6379도 예외 없다). - `cd ~/backend-journey && python3 update.py --done m106`
+ grep 판정 - 걸릴 수 있는 곳: mvn 없다고 하면 `sdk install maven` 한 줄.

4. 검증 질문 (다음 복기용 — 답 적지 말 것)

- 1. 오늘 완결점을 "계정 생성"이 아니라 "clone 명령 확보"로 잡은 이유는?
- 2. 통째 이월과 착수 조각의 차이는? 쪼개는 기준이 된 질문은 무엇이었나?
- 3. CodeCrafters push가 GitHub 잔디에 안 찍히는 이유와, m106 기간의 잔디 경로는?
- 4. 내일 push에서 브랜치 이름을 뭘로 써야 하나? 왜?
- 5. 내일 20분 플랜의 끊는 지점은 어디이고, 그것이 완결 조각인 근거(확립된 스타일)는?

5. 회고

수련회 첫날이라 오늘은 원래 "못 하는 날"이었다. 근데 못 하는 날에도 종류가 있다 — 통째로 미루는 날과, 맨 앞 조각만 떼는 날. 오늘 후자를 골랐고, 고른 기준이 남는다: 오늘 몇 분 했느냐가 아니라 **내일이 어디서 시작하느냐**. 예상표에서 미지수가 제일 큰 구간(모바일 UI 탐색)이 하필 맨 앞이었으니, 그것만 오늘 치우면 내일은 전 구간이 익숙한 터미널이다. 5분짜리 조각이 내일 세션의 질을 바꾼다.

기록도 조각 크기 그대로다. 7/16처럼 — 작은 날을 크게 부풀리면 기록 전체의 신뢰가 죽는다. 내일은 clone 한 줄에서 시작해서, 시간이 되면 6379를 내 손으로 연다. accept() — "기다림"의 정체를 코드로 처음 보는 날이 된다.

한 줄: 수련회 첫날, m106을 통째 이월하는 대신 맨 앞 조각(계정+clone 명령 확보)만 떼서 완결 — 내일의 최대 미지수를 오늘 제거했고, 내일은 터미널에서 시작한다.